

Luca_prezzo.R

Utente

2026-06-06

```
rm(list=ls());gc();

##          used (Mb) gc trigger (Mb) max used (Mb)
## Ncells  584223 31.3   1306308 69.8   926840 49.5
## Vcells 1112944  8.5    8388608 64.0  1928164 14.8

setwd("C:/Users/Utente/OneDrive/Universita/Magistrale/2025-2026/Aziendali/Progetto/Analisi")

load("dati_puliti.Rdata")

source("C:/Users/Utente/OneDrive/Universita/Magistrale/2025-2026/Aziendali/Labs/Lab7/utils.R")

## Warning: il pacchetto 'dplyr' è stato creato con R versione 4.5.3
##
## Caricamento pacchetto: 'dplyr'
## I seguenti oggetti sono mascherati da 'package:stats':
##
##   filter, lag
## I seguenti oggetti sono mascherati da 'package:base':
##
##   intersect, setdiff, setequal, union
## Warning: il pacchetto 'partykit' è stato creato con R versione 4.5.3
## Caricamento del pacchetto richiesto: grid
## Caricamento del pacchetto richiesto: libcoin
## Warning: il pacchetto 'libcoin' è stato creato con R versione 4.5.3
## Caricamento del pacchetto richiesto: mvtnorm
## Warning: il pacchetto 'mvtnorm' è stato creato con R versione 4.5.3
MSE <- function(previsioni, veri_valori){
  mean( (previsioni - veri_valori)^2 )
}

plot_tree <- function(albero, file_name = NULL){
  # Se viene fornito un nome file, attiva il dispositivo corretto
  if(!is.null(file_name)){
    png(file = paste0(file_name, ".png"), width = 1800, height = 1200, res = 200)
  }

  # Logica del grafico (immutata)
```

```

plot(albero, col = "gray40", lwd = 1.8)
text(albero, pretty = 4, digits = 3, cex = 0.8, col = "darkblue", font = 2)

if(!is.null(file_name)) {
  dev.off()
}
dev.off()
}

dataset_regressione_prezzo <-
dataset_regressione_prezzo |>
  rename(Google_Trends = avg_interest_global_last_year,
         Log_Review_Count = Log_Seller_Reviews_Count,
         Item_Verification = Has_Item_Verification) |>
  dplyr::select(-Item_Verification, -Is_Boosted)

nomi_presentazione <- c(
  "Size" = "Taglia",
  "Condition" = "Condizioni dell'Articolo",
  "Material" = "Materiale",
  "Favorites_Count" = "Numero di Preferiti",
  "Shipping_Cost" = "Costo di Spedizione",
  "Color_new" = "Colore",
  "Seller_Rating_Class" = "Fascia Recensione Venditore",
  "Log_Review_Count" = "Log(N° Recensioni)",
  "Seller_has_distintivi" = "Venditore con Badge",
  "Num_Other_Items" = "Altri Articoli in Vendita",
  "Google_Trends" = "Interesse (Google Trends)"
)

# Regressione sul prezzo ----

stima <- dataset_regressione_prezzo[id_stima,]
verifica <- dataset_regressione_prezzo[id_verifica,]

X_stima <- model.matrix(y~., stima)[,-1] |> as.data.frame()
X_verifica <- model.matrix(y~., verifica)[,-1] |> as.data.frame()

colnames(X_stima) <- colnames(X_stima) |>
  gsub(pattern=" ", replacement="_")
colnames(X_verifica) <- colnames(X_stima) |>
  gsub(pattern=" ", replacement="_")

set.seed(1)
idx_a <- sample(1:nrow(X_stima), ceiling(NROW(X_stima)*2/3))
idx_b <- setdiff(1:nrow(X_stima), idx_a)

## CART ----

library(tree)
m_tree_full <- tree(y~., data=stima,
                   split='deviance',
                   control=

```

```

        tree.control(nobs=NROW(stima),
                     minsize=1,
                     mindev=0.0001))
sum(m_tree_full$frame$var == "<leaf>")

## [1] 327

max_size <- 50
K_cv <- 20
set.seed(1)
fold <- sample(1:K_cv, NROW(stima), replace = T)
matrice_errori_albero = matrix(nrow=K_cv, ncol=max_size)
rownames(matrice_errori_albero) <- paste("fold", 1:K_cv)
colnames(matrice_errori_albero) <- 2:(max_size+1)
#for (j in 1:K_cv){
#  id_stima = which(fold != j)
#  id_verifica = which(fold == j)
#
#  m_tree_tmp = tree(y~.,
#                    data=stima[id_stima,],
#                    split='deviance',
#                    control=
#                      tree.control(nobs=NROW(stima),
#                                   minsize=1,
#                                   mindev=0.0001))
#  m_tree_tmp
#
#  tree_list = lapply(2:(max_size+1), # per noi dopo i 50 split é sovra-adattato
#                    # non uso il numero di split m_tree_full perché può
#                    # essere che alcuni alberi siano più piccoli
#                    function(l) prune.tree(m_tree_tmp,
#                                             best=l) )
#  pred_list = lapply(tree_list,
#                    function(x) predict(x,
#                                         stima[id_verifica,]))
#  lista_errori_CI = lapply(pred_list,
#                    function(x) MSE(x, stima$y[id_verifica]))
#
#  matrice_errori_albero[j,] = lista_errori_CI |> unlist()
#}
#save(matrice_errori_albero, file="Modelli_stimati/err_tree_prezzo.RData")
load("Modelli_stimati/err_tree_prezzo.RData")
dim(matrice_errori_albero)

## [1] 20 50

errore_albero_CI = apply(matrice_errori_albero, 2, mean)
se_albero <- apply(matrice_errori_albero, 2, function(x) sd(x) / sqrt(max(K_cv)))

B = which.min(errore_albero_CI) |> names() |> as.numeric()

cv_min <- errore_albero_CI[B]
se_min <- se_albero[B]

soglia_1se <- cv_min + se_min

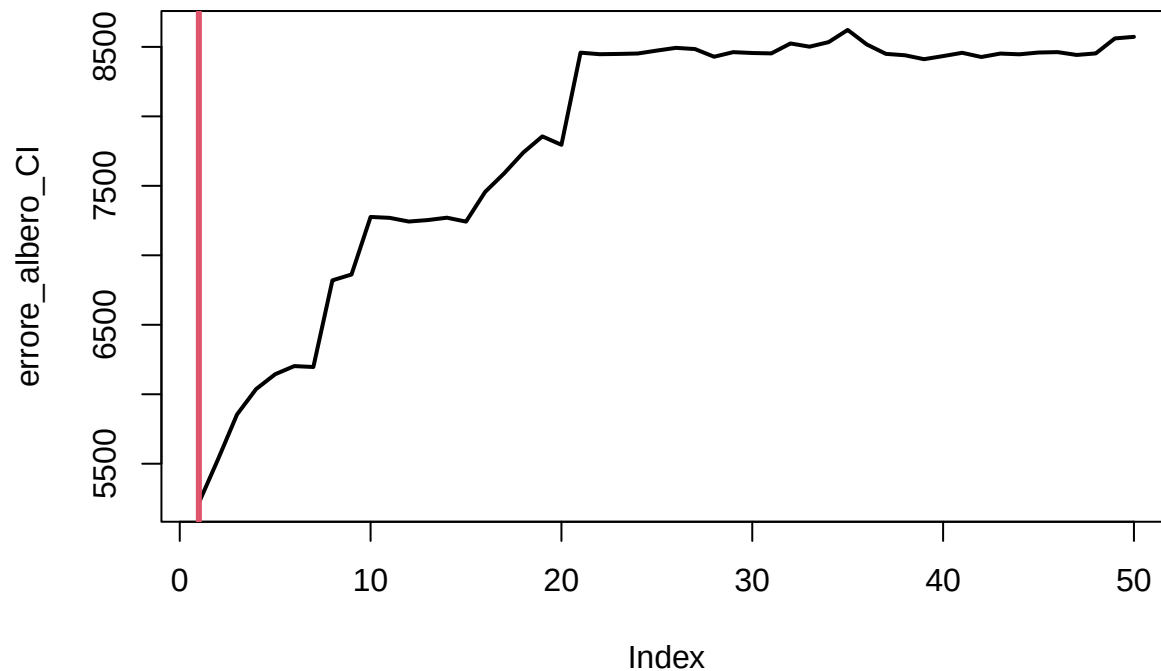
```

```

taglie_valide <- which(errore_albero_CI <= soglia_1se)
B_1se <- max(2, min(taglie_valide))

plot(errore_albero_CI, type='l', lwd=2)
abline(v=B-1, col=2, lwd=3)

```



```

m_tree = prune.tree(m_tree_full, best=B_1se)

#x11();plot_tree(m_tree, "Plot/albero_prezzo")

yhat_tree <- predict(m_tree, newdata = verifica)
err_tree <- mean((verifica$y - yhat_tree)^2)

prezzo_medio_verifica <- mean(verifica$y)
devianza_totale_verifica <- sum((verifica$y - prezzo_medio_verifica)^2)
devianza_residua_albero <- sum((verifica$y - yhat_tree)^2)

r2_tree <- 1 - (devianza_residua_albero / devianza_totale_verifica)

r2_prezzo <- data.frame(albero = r2_tree)
r2_prezzo

##      albero
## 1 0.08226266

rmse_albero <- sqrt(mean((verifica$y - yhat_tree)^2))
rmse_prezzo <- data.frame(albero = rmse_albero)
rmse_prezzo

##      albero
## 1 51.76913

```

```

mae_albero <- mean(abs(verifica$y - yhat_tree))
mae_prezzo <- data.frame(albero = mae_albero)
mae_prezzo

##      albero
## 1 37.49173

## Random forest -----

n_threads <- parallel::detectCores()
library(ranger)

## Warning: il pacchetto 'ranger' è stato creato con R versione 4.5.3
NCOL(stima) |> sqrt(); NCOL(stima) |> log();

## [1] 3.605551
## [1] 2.564949

mtry_values <- c(2,4,6,8,10)
num_tree_values <- c(50, 100, 200, 300, 400)
griddina <- expand.grid(mtry=mtry_values,
                      n_tree=num_tree_values)
modelli_rf <- list()
set.seed(1)
for(i in 1:NROW(griddina)){
  cat(i, "su", NROW(griddina), "\n\n")
  modelli_rf[[i]] <- ranger(y~., num.trees = griddina[i,2],
                          mtry=griddina[i,1],
                          data=stima, importance="none",
                          num.threads=n_threads)
}

## 1 su 25
##
## 2 su 25
##
## 3 su 25
##
## 4 su 25
##
## 5 su 25
##
## 6 su 25
##
## 7 su 25
##
## 8 su 25
##
## 9 su 25
##
## 10 su 25
##
## 11 su 25
##
## 12 su 25

```

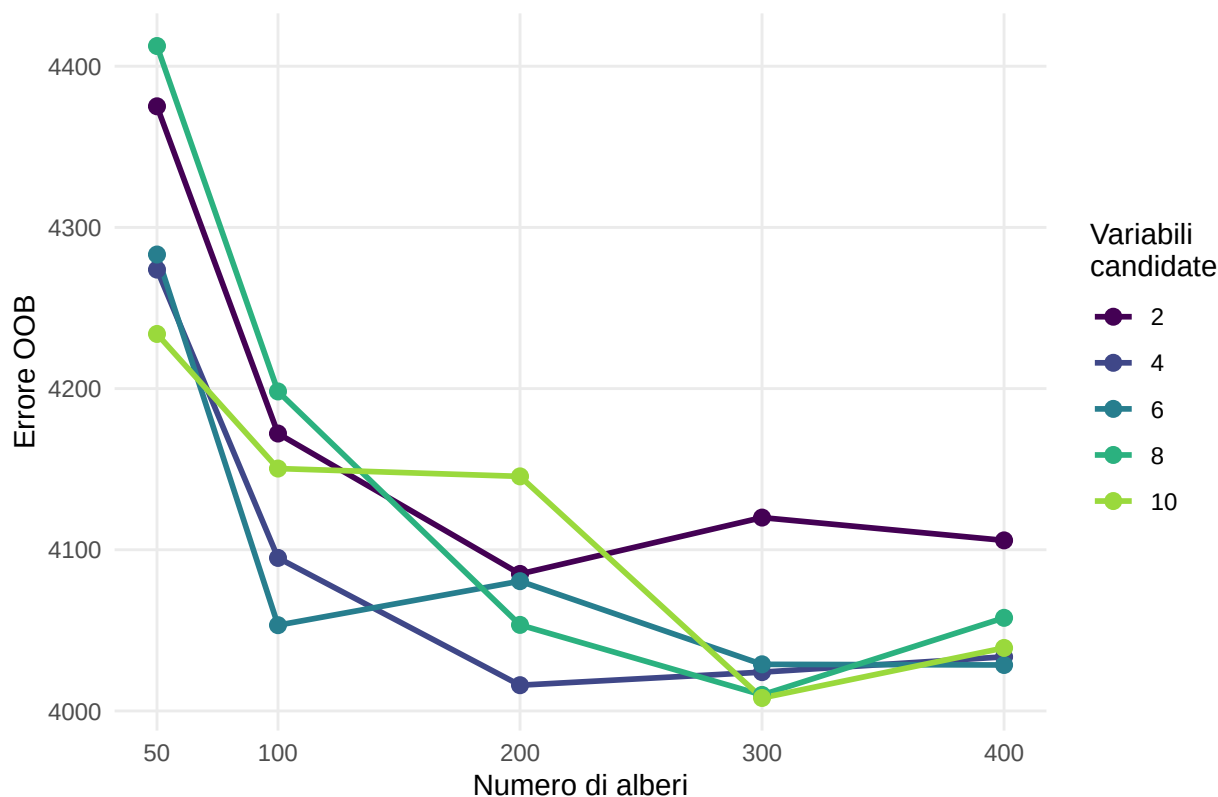
```
##
## 13 su 25
##
## 14 su 25
##
## 15 su 25
##
## 16 su 25
##
## 17 su 25
##
## 18 su 25
##
## 19 su 25
##
## 20 su 25
##
## 21 su 25
##
## 22 su 25
##
## 23 su 25
##
## 24 su 25
##
## 25 su 25

save(modelli_rf, file="Modelli_stimati/err_rf_prezzo.RData")
load("Modelli_stimati/err_rf_prezzo.RData")

errori_OOB <- vapply(modelli_rf, function(x) x$prediction.error, FUN.VALUE = numeric(1))
datini <- data.frame(n_tree = griddina$n_tree,
  mtry = as.factor(griddina$mtry), errore_OOB = errori_OOB)

library(ggplot2)
ggplot(datini, aes(x = n_tree, y = errore_OOB, color = mtry, group = mtry)) +
  geom_line(linewidth = 1) +
  geom_point(size = 2.5) +
  scale_color_viridis_d(option = "viridis", end = 0.85) +
  scale_x_continuous(breaks = num_tree_values) +
  theme_minimal(base_size = 11) +
  labs(
    title = "Tuning Random Forest: Errore Out-of-Bag",
    x = "Numero di alberi",
    y = "Errore OOB",
    color = "Variabili\ncandidate"
  ) +
  theme(plot.title = element_text(face = "bold", size = 13),
    panel.grid.minor = element_blank())
```

Tuning Random Forest: Errore Out-of-Bag



```
ntree_rf <- 300
set.seed(123)
rf_best <- ranger(y~.,
                  num.trees = ntree_rf, mtry = 8,
                  data=stima, importance="permutation",
                  num.threads=n_threads)

p_rf = predict(rf_best, verifica)
devianza_residua_rf <- sum((verifica$y - p_rf$predictions)^2)

r2_rf <- 1 - (devianza_residua_rf / devianza_totale_verifica)

r2_prezzo$RF <- r2_rf
r2_prezzo
```

```
##      albero      RF
## 1 0.08226266 0.2275607
```

```
rmse_rf <- sqrt(mean((verifica$y - p_rf$predictions)^2))
rmse_prezzo$RF <- rmse_rf
rmse_prezzo
```

```
##      albero      RF
## 1 51.76913 47.49455
```

```
mae_rf <- mean(abs(verifica$y - p_rf$predictions))
mae_prezzo$RF <- mae_rf
mae_prezzo
```

```

##      albero      RF
## 1 37.49173 29.18396
### Importanza ----

top_imp <- sort(rf_best$variable.importance, decreasing = TRUE) |> head(10)

tipo_var <- function(nomi) {
  # Classifico manualmente le variabili in 3 gruppi semantici
  dplyr::case_when(
    grepl("Seller|Rating|Reviews|distintivi|Num_Other", nomi) ~ "Venditore",
    grepl("Brand|Size|Condition|Material|Color|avg_interest", nomi) ~ "Prodotto",
    TRUE ~ "Transazione"
  )
}

df_imp <- data.frame(
  variabile = names(top_imp),
  importanza = as.numeric(top_imp)
) |>
  mutate(
    variabile = factor(variabile, levels = rev(variabile)),
    tipo = tipo_var(as.character(variabile))
  )

p_imp_rf <- df_imp %>%
  mutate(
    Nome_Bello = coalesce(nomi_presentazione[variabile], variabile),
    Nome_Bello = forcats::fct_reorder(Nome_Bello, importanza)
  ) |>
  ggplot(aes(x = Nome_Bello, y = importanza, fill = tipo)) +
  geom_col(width = 0.7) +
  coord_flip() +
  scale_fill_manual(
    values = c("Venditore" = "#4E79A7", "Prodotto" = "#F28E2B",
              "Transazione" = "#59A14F"),
    name = "Categoria"
  ) +
  labs(title = "Importanza delle top 10 variabili tramite permutazione",
       x = NULL, y = "Importanza") +
  theme_minimal(base_size = 12) +
  theme(plot.title = element_text(face = "bold", size = 14),
        legend.position = "bottom")

ggsave("Plot/Importanza_RF_prezzo.png", plot=p_imp_rf,
       width = 8, height = 5)

## GBM -----

nus <- c(0.005, 0.01, 0.05, 0.1) # shrinkage
dps <- c(22, 25, 28, 31, 34) # profondità

tuning_par <- expand.grid(nus, dps)
err_gbm <- rep(1, nrow(tuning_par))

```

```

library(gbm)

## Warning: il pacchetto 'gbm' è stato creato con R versione 4.5.3

## Loaded gbm 2.2.3

## This version of gbm is no longer under development. Consider transitioning to gbm3, https://github.com
nts_xgb <- 2000
set.seed(123)
# modello di prova per capire se uso un numero corretto di alberi
#mod_prova <- gbm(stima$y~., data=X_stima,
#                 distribution = "gaussian", n.trees = nts_xgb,
#                 shrinkage = tuning_par[1,1],
#                 interaction.depth = tuning_par[1,2])
#
#yhat_mp <- predict(mod_prova, newdata = X_verifica, n.trees=1:nts_xgb)
#err_pr <- apply(yhat_mp, 2, function(p) mean((verifica$y - p)^2))
#plot(err_pr, type="l")
#abline(h=min(err_pr), col=2)
#abline(v=which.min(err_pr), col=2, lty=2, lwd=2)
#abline(v=100, col=3, lty=2, lwd=2)

#err_pr[450] / err_pr[100]
#err_pr[which.min(err_pr)] / err_pr[100]

nts_xgb <- 800

### Tuning ----

#set.seed(1)
#for(i in 1:nrow(tuning_par)){
#  mod <- gbm(stima$y[idx_a]~., data=X_stima[idx_a,],
#             distribution = "gaussian",
#             shrinkage = tuning_par[i,1], n.trees = nts_xgb,
#             interaction.depth = tuning_par[i,2])
#  passo <- max(30, 1000 * (0.5 - tuning_par[i,1])/(dps/2))
#  alberi_pred <- seq(100, nts_xgb, by = passo)
#  ps <- predict(mod, newdata = X_stima[idx_b,], n.trees = alberi_pred)
#  err_i <- apply(ps, 2, function(p) mean(((stima$y)[idx_b] - p)^2))
#  err_gbm[i] <- min(err_i)
#  cat(round(i/nrow(tuning_par)*100,2), "%\n")
#}
#save(err_gbm, file="Modelli_stimati/err_gbm_prezzo.RData")
load("Modelli_stimati/err_gbm_prezzo.RData")
prezzo_medio_idxb <- mean(stima$y[idx_b])
devianza_totale_idxb <- sum((stima$y[idx_b] - prezzo_medio_idxb)^2)

err_gbm <- 1 - (err_gbm*length(idx_b)) / devianza_totale_idxb

zmat <- matrix(err_gbm, nrow = length(nus), ncol = length(dps), byrow = TRUE)

par_opt <- tuning_par[which.max(err_gbm),] # un parametro di shrinkage troppo elevato mi fa
par_opt

```

```
##      Var1 Var2
## 15 0.05   31

cbind(tuning_par, err_gbm)

##      Var1 Var2   err_gbm
## 1  0.005   22 0.3875326
## 2  0.010   22 0.3875478
## 3  0.050   22 0.3734788
## 4  0.100   22 0.3387452
## 5  0.005   25 0.3909600
## 6  0.010   25 0.3975281
## 7  0.050   25 0.3705096
## 8  0.100   25 0.3586703
## 9  0.005   28 0.3933176
## 10 0.010   28 0.3980278
## 11 0.050   28 0.4056677
## 12 0.100   28 0.3266220
## 13 0.005   31 0.3895194
## 14 0.010   31 0.3861476
## 15 0.050   31 0.4067576
## 16 0.100   31 0.3266981
## 17 0.005   34 0.3913013
## 18 0.010   34 0.4021900
## 19 0.050   34 0.3833384
## 20 0.100   34 0.3339855

risultati_tuning <- data.frame(tuning_par)
colnames(risultati_tuning) <- c("shrinkage", "depth")
risultati_tuning$err_gbm <- err_gbm # Aggiungiamo l'errore calcolato nel ciclo

library(ggplot2)
library(viridis)

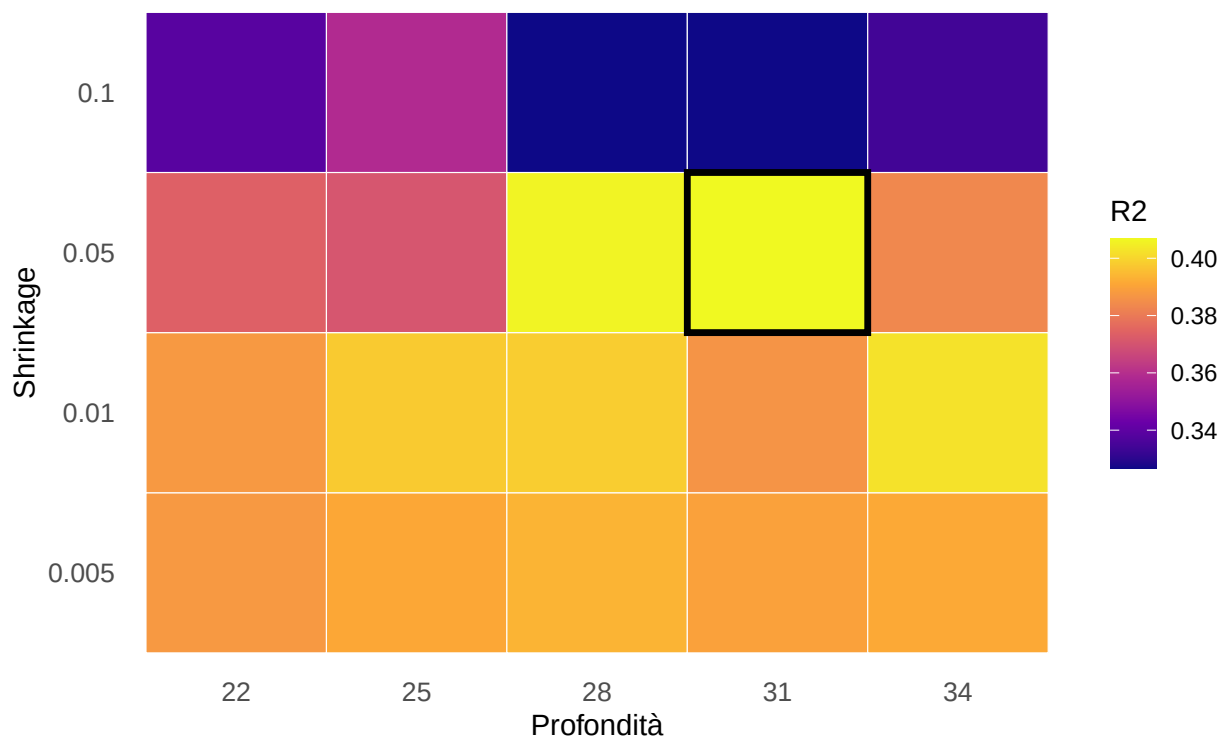
## Warning: il pacchetto 'viridis' è stato creato con R versione 4.5.2
## Caricamento del pacchetto richiesto: viridisLite

ggplot(risultati_tuning, aes(x = factor(depth), y = factor(shrinkage))) +
  geom_tile(aes(fill = err_gbm), color = "white", linewidth = 0.2) +
  geom_tile(data = risultati_tuning[which.max(risultati_tuning$err_gbm), ],
            aes(x = factor(depth), y = factor(shrinkage)),
            fill = NA, color = "black", linewidth = 1.2) +
  scale_fill_viridis_c(option = "plasma", direction = 1, name = "R2") +
  labs(
    title = "Tuning dei Parametri GBM",
    subtitle = "Il riquadro nero indica la combinazione con l'R2 massimo",
    x = "Profondità",
    y = "Shrinkage"
  ) +
  theme_minimal() +
  theme(
    plot.title = element_text(face = "bold", size = 14),
    plot.subtitle = element_text(color = "gray40", size = 10),
    axis.text = element_text(size = 10),
    panel.grid = element_blank()
  )
```

)

Tuning dei Parametri GBM

Il riquadro nero indica la combinazione con l'R2 massimo



```
ggsave("Plot/tuning_gbm_prezzo.png")
```

```
## Saving 6.5 x 4.5 in image
```

```
### Fit finale ----
```

```
set.seed(123)
mod_gbm <- gbm(stima$y[idx_a]~., data=X_stima[idx_a,],
               n.trees = nts_xgb, distribution = "gaussian",
               shrinkage = par_opt[1],
               interaction.depth = par_opt[2])

ps <- predict(mod_gbm, newdata = X_stima[idx_b,],
              n.trees = seq(60, nts_xgb, by=20))
err_i <- apply(ps, 2, function(p) mean((stima$y[idx_b] - p)^2))
n_trees_best <- names(err_i)[which.min(err_i)] |> as.numeric()
n_trees_best
```

```
## [1] 140
```

```
yhat_gbm <- predict(mod_gbm, newdata = X_verifica,
                   n.trees = n_trees_best)
devianza_residua_gbm <- sum((verifica$y - yhat_gbm)^2)

r2_gbm <- 1 - (devianza_residua_gbm / devianza_totale_verifica)

r2_prezzo$GBM <- r2_gbm
```

```

r2_prezzo

##      albero      RF      GBM
## 1 0.08226266 0.2275607 0.2015073

rmse_gbm <- sqrt(mean((verifica$y - yhat_gbm)^2))
rmse_prezzo$gbm <- rmse_gbm
rmse_prezzo

##      albero      RF      gbm
## 1 51.76913 47.49455 48.28888

mae_gbm <- mean(abs(verifica$y - yhat_gbm))
mae_prezzo$gbm <- mae_gbm
mae_prezzo

##      albero      RF      gbm
## 1 37.49173 29.18396 31.55931

colnames(X_stima)

## [1] "Brandasics"
## [2] "Brandastry"
## [3] "BrandMarche_di_Lusso"
## [4] "Brandchampion"
## [5] "Brandconverse"
## [6] "Branddiadora"
## [7] "Brandfila"
## [8] "Brandignoto"
## [9] "Brandnew_balance"
## [10] "Brandnike"
## [11] "Brandpuma"
## [12] "Brandreebok"
## [13] "Brandvans"
## [14] "BrandAltro"
## [15] "Size"
## [16] "Condition.L"
## [17] "Condition.Q"
## [18] "Condition.C"
## [19] "Condition^4"
## [20] "Materialnoto"
## [21] "Favorites_Count"
## [22] "Shipping_Cost"
## [23] "Color_newColorato"
## [24] "Color_newMulticolor"
## [25] "Color_newScuro"
## [26] "Seller_Rating_ClassBasso_(<=4.0)"
## [27] "Seller_Rating_ClassBuono_(4.1_-4.6)"
## [28] "Seller_Rating_ClassOttimo_(4.7_-4.9)"
## [29] "Seller_Rating_ClassPerfetto_(5.0)"
## [30] "Log_Review_Count"
## [31] "Seller_has_distintivi"
## [32] "Num_Other_Items"
## [33] "Google_Trends"

```

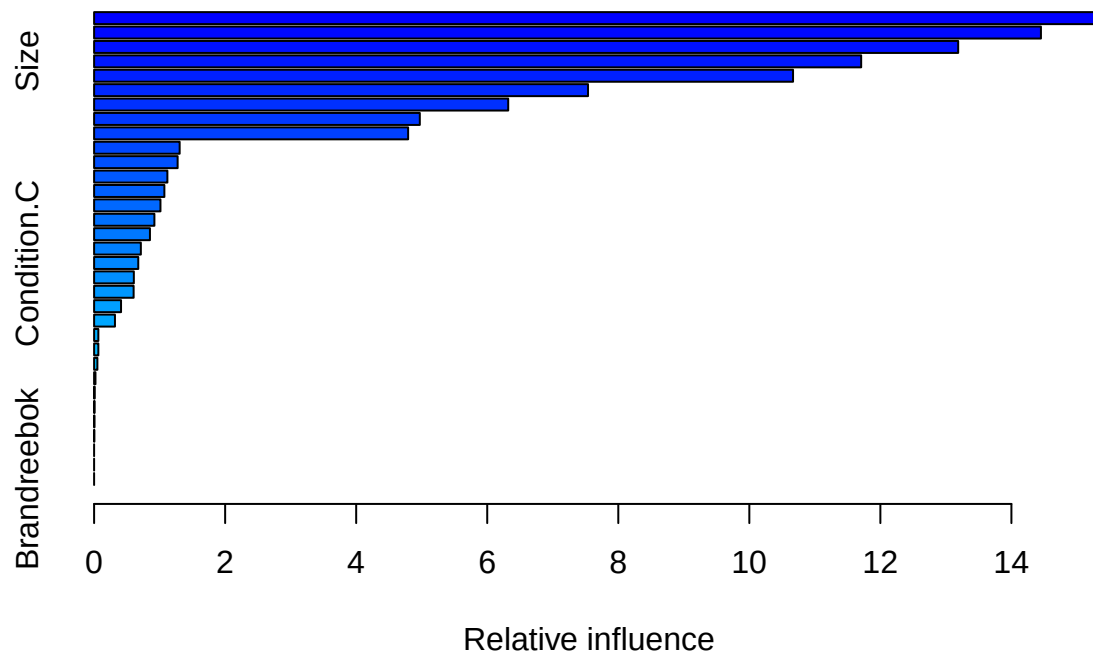
```
dim(X_stima)
```

```
## [1] 1434 33
```

```
## PDP ----
```

```
# Trovo le top3 variabili per devianza spiegata
```

```
top3_gbm <- summary(mod_gbm, n.trees = n_trees_best,  
                    plotit = T, order = TRUE)$var[c(2,3,6)]
```



```
# Indici colonna delle top 3 variabili
```

```
idx_top3 <- sapply(top3_gbm, function(v) which(colnames(X_stima) == v))
```

```
# Definisco la funzione per calcolare i dati del Partial Dependence Plot singolo
```

```
pdp_gbm_single <- function(var_idx, modello, X_data, n_trees, dati_orig,  
                             n_grid = 50) {
```

```
  # Estraggo il nome della variabile, i dati e verifico se è numerica
```

```
  var_name <- colnames(X_data)[var_idx]
```

```
  x <- X_data[[var_name]]
```

```
  is_num <- is.numeric(x)
```

```
  # Creo la griglia: sequenza per i numerici (escludendo le code 2%-98%), categorie uniche altrimenti
```

```
  griglia <- if (is_num)
```

```
    seq(quantile(x, 0.02), quantile(x, 0.98), length.out = n_grid)
```

```
  else
```

```
    sort(unique(x))
```

```
  # Calcolo la predizione media per ogni valore della griglia
```

```
  pds <- sapply(griglia, function(v) {
```

```
    tmp <- X_data
```

```
    tmp[[var_name]] <- v
```

```
    mean(predict(modello, newdata = tmp, n.trees = n_trees))
```

```
  })
```

```

# Restituisco i risultati impaginati in un dataframe
data.frame(x = griglia, pd = pds, variabile = var_name, is_num = is_num)
}

# Calcolo i dati PDP per le top 3 variabili applicando la funzione
pdp_gbm_list <- lapply(idx_top3, pdp_gbm_single,
                        modello = mod_gbm,
                        X_data = X_stima,
                        n_trees = n_trees_best,
                        dati_orig = stima)

# Genero la lista dei 3 grafici ggplot
pdp_gbm_plots <- lapply(pdp_gbm_list, function(df) {

  nome_brutto <- df$variabile[1]
  nome_bello <- coalesce(nomi_presentazione[nome_brutto], nome_brutto)

  # Inizializzo il grafico gestendo l'asse x in base al tipo di variabile
  p <- ggplot(df, aes(x = if (df$is_num[1]) as.numeric(x) else factor(x),
                      y = pd))

  # Aggiungo la geometria adeguata: linea se numerico, barre se categorico
  if (df$is_num[1]) {
    p <- p + geom_line(color = "#E15759", linewidth = 1.2)
  } else {
    p <- p + geom_col(fill = "#E15759", width = 0.6)
  }

  # Formatto i titoli e il tema estetico
  p + labs(title = nome_bello, x = NULL, y = "Prezzo atteso (€)") +
    theme_minimal(base_size = 11) +
    theme(plot.title = element_text(face = "bold"))
})

library(patchwork)

## Warning: il pacchetto 'patchwork' è stato creato con R versione 4.5.3

# Unisco i tre grafici affiancati e aggiungo un titolo globale
p_pdp_gbm <- (pdp_gbm_plots[[1]] | pdp_gbm_plots[[2]] | pdp_gbm_plots[[3]]) +
  plot_annotation(
    theme = theme(plot.title = element_text(face = "bold", size = 14))
  )

ggsave("Plot/gbm_pdp_top3_prezzo.png",
        plot = p_pdp_gbm, width = 12, height = 5, dpi = 200)

```

```

## Warning: Use of `df$is_num` is discouraged.
## i Use `is_num` instead.

## Warning: Use of `df$is_num` is discouraged.
## i Use `is_num` instead.
## Use of `df$is_num` is discouraged.
## i Use `is_num` instead.

```

```

## Effetti casuali -----

### MERT -----

form <- formula(lm(y~.,
                    data=stima |> dplyr::select(-Brand,
                                                -Condition)))

set.seed(123)
mert_model <- MERT(formula = form , random = " + (1|Brand) + (1|Condition)",
                    data = stima, cv=T)

## Caricamento del pacchetto richiesto: rpart
## Warning: il pacchetto 'rpart' è stato creato con R versione 4.5.3
## Caricamento del pacchetto richiesto: lme4
## Warning: il pacchetto 'lme4' è stato creato con R versione 4.5.2
## Caricamento del pacchetto richiesto: Matrix
## Warning: il pacchetto 'Matrix' è stato creato con R versione 4.5.3

## [1] "Log likelihood: -8126.68900527948"
## [1] "Log likelihood: -8126.68621041771"
## [1] "Log likelihood: -8126.68348728402"
## [1] "Log likelihood: -8126.68083410386"
## [1] "Log likelihood: -8126.67824914266"
## [1] "Log likelihood: -8126.67573070314"
## [1] "Log likelihood: -8126.67327713164"
## [1] "Log likelihood: -8126.67088681355"
## [1] "Log likelihood: -8126.66855815839"
## [1] "Log likelihood: -8126.6662896291"
## [1] "Log likelihood: -8126.66407971285"
## [1] "Log likelihood: -8126.66192693731"
## [1] "Log likelihood: -8126.65982986076"
## [1] "Log likelihood: -8126.65778707399"
## [1] "Log likelihood: -8126.65579722726"
## [1] "Log likelihood: -8126.65385892231"
## [1] "Log likelihood: -8126.65197092831"
## [1] "Log likelihood: -8126.65013189522"
## [1] "Log likelihood: -8126.64834065861"
## [1] "Log likelihood: -8126.64659595675"
## [1] "Log likelihood: -8126.64489660105"
## [1] "Log likelihood: -8126.64324144104"
## [1] "Log likelihood: -8126.64162934603"
## [1] "Log likelihood: -8126.64005921163"
## [1] "Log likelihood: -8126.63852997127"
## [1] "Log likelihood: -8126.63704056635"
## [1] "Log likelihood: -8126.63558999208"
## [1] "Log likelihood: -8126.6341771811"
## [1] "Log likelihood: -8126.6328012772"
## [1] "Log likelihood: -8126.63146127527"
## [1] "Log likelihood: -8126.63015627563"
## [1] "Log likelihood: -8126.62888527343"
## [1] "Log likelihood: -8126.62764751675"

```

```
## [1] "Log likelihood: -8126.62644210199"
## [1] "Log likelihood: -8126.62526818848"
## [1] "Log likelihood: -8126.62412495877"
## [1] "Log likelihood: -8126.62301182227"
## [1] "Log likelihood: -8126.62192751663"
## [1] "Log likelihood: -8126.62087166194"
## [1] "Log likelihood: -8126.61984345755"
## [1] "Log likelihood: -8126.6188421133"
## [1] "Log likelihood: -8126.61786700609"
```

```
options(max.print=100)
mert_model
```

```
## $Tree
## n= 1434
##
## node), split, n, deviance, yval
##      * denotes terminal node
##
## 1) root 1434 6637909 52.15707 *
##
## $Tree_np
## n= 1434
##
## node), split, n, deviance, yval
##      * denotes terminal node
##
##      1) root 1434 6637909.000  52.1570700
##          2) Size< 38.75 625  962292.400  36.2381200
##              4) Size< 35.25 176  62218.190  17.1053900
##                  8) Google_Trends< 18.78704 13  1543.584  -4.8370920 *
##                  9) Google_Trends>=18.78704 163  53916.270  18.8554000
##                      18) Google_Trends>=22.98148 150  40302.930  16.3963200 *
##                      19) Google_Trends< 22.98148 13  2240.156  47.2294400 *
##          5) Size>=35.25 449  810393.200  43.7378100
##              10) Google_Trends< 60.07407 388  414490.000  40.6031300
##                  20) Google_Trends< 17.02778 36  41769.190  21.6738900
##                      40) Material=ignoto 26  13786.500  11.8187600 *
##                      41) Material=noto 10  18891.930  47.2972200 *
##          21) Google_Trends>=17.02778 352  358502.200  42.5390800
##              42) Google_Trends>=38.12963 141  124009.500  34.3479300
##                  84) Google_Trends< 44 60  30913.980  17.2235000 *
##                  85) Google_Trends>=44 81  62467.570  47.0327000 *
##          43) Google_Trends< 38.12963 211  218710.500  48.0127800
##              86) Google_Trends< 33.23606 81  100392.900  37.5301100
##                  172) Google_Trends>=24.19444 44  18984.280  25.5688900 *
##                  173) Google_Trends< 24.19444 37  67627.460  51.7542700
##                      346) Shipping_Cost< 3.255 30  10718.640  42.6010100 *
##                      347) Shipping_Cost>=3.255 7  43623.350  90.9825400 *
##          87) Google_Trends>=33.23606 130  103870.900  54.5442900
##              174) Log_Review_Count>=5.339798 29  6480.735  37.1497700 *
##              175) Log_Review_Count< 5.339798 101  86096.200  59.5387600
##                  350) Log_Review_Count< 5.036932 94  55952.350  57.3170700 *
##                  351) Log_Review_Count>=5.036932 7  23449.360  89.3728700 *
##          11) Google_Trends>=60.07407 61  367840.200  63.6764100
```

```

##      22) Google_Trends>=61.35185 53 122798.800 42.2928200
##      44) Google_Trends< 67.94444 8 1101.516 1.0583960 *
##      45) Google_Trends>=67.94444 45 105676.900 49.6233900
##      90) Favorites_Count>=3.5 37 29311.810 39.4937700 *
##      91) Favorites_Count< 3.5 8 55009.530 96.4728700 *
##      23) Google_Trends< 61.35185 8 60251.970 205.3427000 *
## 3) Size>=38.75 809 5394873.000 64.4554000
##      6) Google_Trends< 73.69444 796 4393863.000 62.2104100
##      12) Shipping_Cost< 2.97 431 1564193.000 49.6645600
##      24) Num_Other_Items>=16.5 224 622781.500 37.1444500
##      48) Google_Trends< 20.80556 26 43564.060 11.8430500
##      96) Favorites_Count>=0.5 18 10429.320 -0.4014303 *
##      97) Favorites_Count< 0.5 8 24363.990 39.3931200 *
##      49) Google_Trends>=20.80556 198 560387.700 40.4668500
##      98) Size< 41.75 140 132315.900 34.9594400 *
##      99) Size>=41.75 58 413575.400 53.7606000
##      198) Favorites_Count>=1.5 51 115281.600 39.7210300
##      396) Log_Review_Count>=1.700599 43 40393.970 33.0559800
##      792) Log_Review_Count< 3.215666 7 6080.350 -0.6306142 *
##      793) Log_Review_Count>=3.215666 36 24825.550 39.6061500 *
##      397) Log_Review_Count< 1.700599 8 62710.200 75.5456600 *
##      199) Favorites_Count< 1.5 7 215000.900 156.0489000 *
##      25) Num_Other_Items< 16.5 207 868302.100 63.2128900
##      50) Log_Review_Count< 5.551267 198 728287.600 59.9618200
##      100) Google_Trends< 20.80556 10 5156.956 2.6550580 *
##      101) Google_Trends>=20.80556 188 688543.100 63.0100500
##      202) Google_Trends>=56.80556 21 13895.580 27.5542700 *
##      203) Google_Trends< 56.80556 167 644928.500 67.4685600
##      406) Favorites_Count>=17.5 67 50264.830 53.7838900 *
##      407) Favorites_Count< 17.5 100 573710.000 76.6372900
##      814) Favorites_Count< 14.5 93 299416.500 71.0919500
##      1628) Size< 40.75 33 62684.930 45.1605300 *
##      1629) Size>=40.75 60 202336.400 85.3542300
##      3258) Shipping_Cost< 2.45 34 67787.350 64.3274200
##      6516) Shipping_Cost>=0.975 25 22142.930 53.2870800 *
##      6517) Shipping_Cost< 0.975 9 34132.670 94.9950300 *
##      3259) Shipping_Cost>=2.45 26 99859.040 112.8508000
##      6518) Shipping_Cost>=2.62 14 52455.210 79.8843900 *
##      6519) Shipping_Cost< 2.62 12 14437.980 151.3117000 *
##      815) Favorites_Count>=14.5 7 233439.000 150.3110000 *
##      51) Log_Review_Count>=5.551267 9 91881.230 134.7364000 *
## 13) Shipping_Cost>=2.97 365 2681726.000 77.0248400
##      26) Log_Review_Count>=3.749435 171 445933.800 59.4924400
##      52) Google_Trends< 57.75926 144 239837.600 54.5071700
##      104) Log_Review_Count< 6.041438 117 136064.500 47.8411900
##      208) Size< 44.25 108 90641.470 45.4931400 *
##      209) Size>=44.25 9 37682.360 76.0177200 *
##      105) Log_Review_Count>=6.041438 27 76045.470 83.3930700
##      210) Size< 39.25 7 2001.436 36.3073800 *
##      211) Size>=39.25 20 53092.800 99.8730600
##      422) Log_Review_Count>=6.422751 8 15324.170 72.0933500 *
##      423) Log_Review_Count< 6.422751 12 27479.130 118.3929000 *
##      53) Google_Trends>=57.75926 27 183430.300 86.0805400
##      106) Google_Trends>=65.34259 20 2701.516 57.7797300 *

```

```

##          107) Google_Trends< 65.34259 7 118942.400 166.9400000 *
##      27) Log_Review_Count< 3.749435 194 2136899.000 92.4786600
##          54) Favorites_Count>=4.5 102 429684.500 64.2590500
##          108) Size< 44.75 94 228933.000 55.5386000
##          216) Favorites_Count>=18.5 43 54605.570 42.4401700
##          432) Shipping_Cost>=3.285 25 31233.260 31.6310700 *
##          433) Shipping_Cost< 3.285 18 16394.580 57.4527900 *
##          217) Favorites_Count< 18.5 51 160729.700 66.5823700
##          434) Google_Trends>=33.23606 40 49212.570 55.8377500 *
##          435) Google_Trends< 33.23606 11 90106.990 105.6537000 *
##          109) Size>=44.75 8 109610.000 166.7244000 *
##          55) Favorites_Count< 4.5 92 1535930.000 123.7656000
##          110) Size>=39.5 80 788098.200 107.2903000
##          220) Seller_Rating_Class=Ottimo (4.7 - 4.9),Perfetto (5.0) 56 322298.200 82.5934600
##          440) Num_Other_Items>=13.5 23 84706.420 54.4915100
##          880) Favorites_Count< 2.5 16 22823.040 35.7196500 *
##          881) Favorites_Count>=2.5 7 43358.130 97.3986100 *
##          441) Num_Other_Items< 13.5 33 206768.700 102.1797000
##          882) Color_new=Chiaro,Multicolor 21 79511.780 81.8861600
##          1764) Log_Review_Count>=1.700599 12 17580.190 56.8501700 *
##          1765) Log_Review_Count< 1.700599 9 44381.170 115.2675000 *
##          883) Color_new=Scuro 12 103474.000 137.6933000 *
##          221) Seller_Rating_Class=Nessuna recensione,Basso (<= 4.0),Buono (4.1 - 4.6) 24 3519
##          442) Google_Trends>=41.53704 12 241108.400 125.9173000 *
##          443) Google_Trends< 41.53704 12 74335.690 203.9151000 *
##          111) Size< 39.5 12 581350.600 233.6013000 *
##      7) Google_Trends>=73.69444 13 751352.100 201.9174000 *
##
## $EffectModel
## Linear mixed model fit by maximum likelihood ['lmerMod']
## Formula: resid ~ -1 + (1 | Brand) + (1 | Condition)
## Data: newdata
##      AIC      BIC    logLik -2*log(L)  df.resid
## 16259.236 16275.040 -8126.618 16253.236      1431
## Random effects:
## Groups   Name      Std.Dev.
## Brand    (Intercept) 41.46
## Condition (Intercept) 24.62
## Residual              68.47
## Number of obs: 1434, groups: Brand, 15; Condition, 5
## No fixed effect coefficients
##
## $RandomEffects
##              (Intercept)
## adidas              -4.526189
## asics                10.421686
## autry                58.531610
## Marche di Lusso    114.006590
## champion            -39.602630
## converse             -22.224778
## diadora            -31.868695
## fila               -29.520800
## ignoto             -22.984666
## new balance         3.984847

```

```

## nike          14.331604
## puma          -29.002962
## reebok        -28.682343
## vans          -28.608077
## Altro         6.353950
##
## $ErrorVariance
## [1] 4687.835
##
## $RanEffVariance
##      Brand Condition
## 1718.7834 606.0669
##
## $data
## # A tibble: 1,434 x 13
##       y Brand   Size Condition Material Favorites_Count Shipping_Cost Color_new
##   <dbl> <fct> <dbl> <ord>    <fct>          <dbl>         <dbl> <fct>
## 1  49  puma   37.5 Nuovo co~ ignoto           34          3.29 Colorato
## 2 20.0 fila   36  Otttime ignoto            1          2.89 Chiaro
## 3  13  Altro   36  Nuovo se~ ignoto          10          2.29 Multicol~
## 4   6  Altro   39  Otttime ignoto            0          1.49 Multicol~
## 5 160  nike   45  Nuovo co~ ignoto            0          3.15 Scuro
## 6  20  nike   36  Otttime ignoto            4          2.99 Multicol~
## 7   7  Altro   22  Otttime ignoto          11          2.29 Multicol~
## 8  20  adidas  39  Otttime ignoto            8          3.29 Multicol~
## 9 110.  nike   42  Otttime ignoto            2          2.65 Scuro
## 10  5  Altro   28  Nuovo se~ ignoto            6          2.29 Multicol~
## # i 1,424 more rows
## # i 5 more variables: Seller_Rating_Class <fct>, Log_Review_Count <dbl>,
## #   Seller_has_distintivi <dbl>, Num_Other_Items <dbl>, Google_Trends <dbl>
##
## $logLik
## 'log Lik.' -8126.618 (df=3)
##
## $IterationsUsed
## [1] 42
##
## $Formula
## y ~ Size + Material + Favorites_Count + Shipping_Cost + Color_new +
##   Seller_Rating_Class + Log_Review_Count + Seller_has_distintivi +
##   Num_Other_Items + Google_Trends
##
## $Random
## [1] " + (1|Brand) + (1|Condition)"
##
## $ErrorTolerance
## [1] 0.001
##
## $Residuals
##           y
## 1    -4.163837
## 2    -1.259897
## 3   -58.854703
## 4   -51.124647

```

## 5	63.501597
## 6	-45.102300
## 7	-50.124647
## 8	-26.244508
## 9	44.397700
## 10	-66.854703
## 11	-6.102300
## 12	-47.124647
## 13	101.359389
## 14	-15.484457
## 15	-17.102300
## 16	-21.477754
## 17	21.097999
## 18	-3.088354
## 19	8.755492
## 20	-1.168067
## 21	90.167644
## 22	-22.124647
## 23	-52.124647
## 24	-53.124647
## 25	112.213969
## 26	-9.302307
## 27	-1.249897
## 28	-22.640611
## 29	35.514401
## 30	-29.846858
## 31	-24.275975
## 32	-13.767734
## 33	-46.498403
## 34	-49.124647
## 35	-32.856858
## 36	22.911646
## 37	8.232266
## 38	50.514401
## 39	-37.516087
## 40	-36.854703
## 41	-36.498403
## 42	21.359389
## 43	44.897700
## 44	-66.854703
## 45	-31.244508
## 46	55.967637
## 47	30.454081
## 48	33.500503
## 49	-11.698410
## 50	-40.244508
## 51	27.213969
## 52	42.865353
## 53	-11.249897
## 54	68.145297
## 55	49.897700
## 56	-4.755543
## 57	-19.942022
## 58	28.755492

```

## 59 -24.646000
## 60 -4.302307
## 61 -17.640611
## 62 -47.602300
## 63 8.807617
## 64 -55.102300
## 65 -15.112300
## 66 -7.898123
## 67 15.367943
## 68 -17.786031
## 69 -42.588486
## 70 -41.854703
## 71 -57.854703
## 72 -6.767734
## 73 -36.824511
## 74 -6.902001
## 75 19.510055
## 76 -32.124647
## 77 31.053142
## 78 11.755492
## 79 -12.640611
## 80 9.734196
## 81 38.175489
## 82 -11.898123
## 83 -59.102300
## 84 -24.755543
## 85 3.750103
## 86 23.755492
## 87 -78.020750
## 88 -35.846858
## 89 -47.102300
## 90 -28.846858
## 91 -6.974564
## 92 20.153142
## 93 -37.124647
## 94 3.755492
## 95 -52.124647
## 96 9.301590
## 97 -121.499497
## 98 -16.244508
## 99 -27.102300
## 100 -32.124647
## [ reached 'max' / getOption("max.print") -- omitted 1334 rows ]
##
## $REML
## [1] TRUE
##
## $cv
## [1] TRUE
##
## $lmer.control
## $optimizer
## [1] "nloptwrap"
##

```

```

## $restart_edge
## [1] TRUE
##
## $boundary.tol
## [1] 1e-05
##
## $calc.derivs
## NULL
##
## $use.last.params
## [1] FALSE
##
## $checkControl
## $checkControl$autoscale
## NULL
##
## $checkControl$check.nobs.vs.rankZ
## [1] "ignore"
##
## $checkControl$check.nobs.vs.nlev
## [1] "stop"
##
## $checkControl$check.nlev.gtreq.5
## [1] "ignore"
##
## $checkControl$check.nlev.gtr.1
## [1] "stop"
##
## $checkControl$check.nobs.vs.nRE
## [1] "stop"
##
## $checkControl$check.rankX
## [1] "message+drop.cols"
##
## $checkControl$check.scaleX
## [1] "warning"
##
## $checkControl$check.formula.LHS
## [1] "stop"
##
##
## $checkConv
## $checkConv$check.conv.nobsmax
## [1] 10000
##
## $checkConv$check.conv.nparmax
## [1] 10
##
## $checkConv$check.conv.grad
## $checkConv$check.conv.grad$action
## [1] "warning"
##
## $checkConv$check.conv.grad$tol
## [1] 0.002

```

```

##
## $checkConv$check.conv.grad$relTol
## NULL
##
##
## $checkConv$check.conv.singular
## $checkConv$check.conv.singular$action
## [1] "message"
##
## $checkConv$check.conv.singular$tol
## [1] 1e-04
##
##
## $checkConv$check.conv.hess
## $checkConv$check.conv.hess$action
## [1] "warning"
##
## $checkConv$check.conv.hess$tol
## [1] 1e-06
##
##
## $optCtrl
## list()
##
## attr("class")
## [1] "lmerControl" "merControl"
##
## $tree.control
## $tree.control$minsplit
## [1] 20
##
## $tree.control$minbucket
## [1] 7
##
## $tree.control$cp
## [1] 0.01
##
## $tree.control$maxcompete
## [1] 4
##
## $tree.control$maxsurrogate
## [1] 5
##
## $tree.control$usesurrogate
## [1] 2
##
## $tree.control$surrogatestyle
## [1] 0
##
## $tree.control$maxdepth
## [1] 30
##
## $tree.control$xval

```

```
## [1] 10

options(max.print=1000)
summary(mert_model$EffectModel)

## Linear mixed model fit by maximum likelihood ['lmerMod']
## Formula: resid ~ -1 + (1 | Brand) + (1 | Condition)
## Data: newdata
## Control: lmer.control
##
##      AIC      BIC    logLik -2*log(L)  df.resid
## 16259.2 16275.0 -8126.6 16253.2    1431
##
## Scaled residuals:
##      Min       1Q   Median       3Q      Max
## -1.7746 -0.4888 -0.1837  0.1359 10.2658
##
## Random effects:
## Groups      Name      Variance Std.Dev.
## Brand      (Intercept) 1718.8   41.46
## Condition (Intercept)  606.1    24.62
## Residual                    4687.8   68.47
## Number of obs: 1434, groups: Brand, 15; Condition, 5

if (NROW(mert_model$Tree$splits) > 0){
  plot_tree(mert_model$Tree, "Plot/MERT_prezzo")
}

pred_tree <- predict(mert_model$Tree, newdata = verifica)
pred_ran <- predict(mert_model$EffectModel, newdata = verifica)
pred_mert <- pred_tree + pred_ran

devianza_residua_mert <- sum((verifica$y - pred_mert)^2)

r2_mert <- 1 - (devianza_residua_mert / devianza_totale_verifica)

r2_prezzo$mert <- r2_mert
r2_prezzo

##      albero      RF      GBM      mert
## 1 0.08226266 0.2275607 0.2015073 0.2187767

rmse_mert <- sqrt(mean((verifica$y - pred_mert)^2))
rmse_prezzo$mert <- rmse_mert
rmse_prezzo

##      albero      RF      gbm      mert
## 1 51.76913 47.49455 48.28888 47.76384

mae_mert <- mean(abs(verifica$y - pred_mert))
mae_prezzo$mert <- mae_mert
mae_prezzo

##      albero      RF      gbm      mert
## 1 37.49173 29.18396 31.55931 33.2487
```

```
### MERF -----
```

```
str(stima)
```

```
## tibble [1,434 x 13] (S3: tbl_df/tbl/data.frame)
## $ y : num [1:1434] 49 20 13 6 160 ...
## $ Brand : Factor w/ 15 levels "adidas","asics",...: 12 8 15 15 11 11 15 1 11 15 ...
## $ Size : num [1:1434] 37.5 36 36 39 45 36 22 39 42 28 ...
## $ Condition : Ord.factor w/ 5 levels "Discrete"<"Buone"<...: 5 3 4 3 5 3 3 3 3 4 ...
## $ Material : Factor w/ 2 levels "ignoto","noto": 1 1 1 1 1 1 1 1 1 1 ...
## $ Favorites_Count : num [1:1434] 34 1 10 0 0 4 11 8 2 6 ...
## $ Shipping_Cost : num [1:1434] 3.29 2.89 2.29 1.49 3.15 2.99 2.29 3.29 2.65 2.29 ...
## $ Color_new : Factor w/ 4 levels "Chiaro","Colorato",...: 2 1 3 3 4 3 3 3 4 3 ...
## ..- attr(*, "names")= chr [1:1434] "Rosa" "Bianco" "Bianco, Verde" "Rosa, Bianco" ...
## $ Seller_Rating_Class : Factor w/ 5 levels "Nessuna recensione",...: 5 4 5 5 3 4 4 4 4 3 ...
## $ Log_Review_Count : num [1:1434] 5.3 5.84 4.22 1.95 3.69 ...
## $ Seller_has_distintivi: num [1:1434] 1 1 1 0 0 0 1 1 1 1 ...
## $ Num_Other_Items : num [1:1434] 20 10 16 14 19 20 20 20 4 7 ...
## $ Google_Trends : num [1:1434] 36.1 68 39.8 67 43.2 ...
```

```
form <- formula(lm(y~.,
                    data=stima |> dplyr::select(-Brand)))
```

```
set.seed(123)
```

```
merf_model <- MERF_ranger_safe(formula = form, data = stima,
                               random = " + (1|Brand)",
                               num.trees_final = ntree_rf, # stesso di prima
                               num.threads = n_threads-2,
                               mtry_grid = c(2,4,6,8)
)
merf_model$mtry_last
```

```
## [1] 4
```

```
pred_fix <- predict(merf_model$RandomForest, data = verifica)$predictions
pred_ran <- predict(merf_model$EffectModel, newdata = verifica) - predict(merf_model$EffectModel, newda
pred_merf <- pred_fix + pred_ran
```

```
devianza_residua_merf <- sum((verifica$y - pred_merf)^2)
r2_merf <- 1 - (devianza_residua_merf / devianza_totale_verifica)
```

```
r2_prezzo$merf <- r2_merf
r2_prezzo
```

```
##      albero      RF      GBM      mert      merf
## 1 0.08226266 0.2275607 0.2015073 0.2187767 0.2925826
```

```
rmse_merf <- sqrt(mean((verifica$y - pred_merf)^2))
rmse_prezzo$merf <- rmse_merf
rmse_prezzo
```

```
##      albero      RF      gbm      mert      merf
## 1 51.76913 47.49455 48.28888 47.76384 45.45163
```

```
mae_merf <- mean(abs(verifica$y - pred_merf))
mae_prezzo$merf <- mae_merf
```

```
mae_prezzo
```

```
##      albero      RF      gbm      mert      merf
## 1 37.49173 29.18396 31.55931 33.2487 28.94816
```

```
### Metboost -----
```

```
#x_var <- colnames(stima |> dplyr::select(-y, -Brand))
#set.seed(1)
#par_opt # del GBM classico
#met_model <- metboost_fit_path_manual(stima, y_name = "y", vars_x = x_var,
#                                     group_var = "Brand",
#                                     #M_max = nts_xgb,
#                                     M_max = 300,
#                                     shrinkage = 0.01, depth = 4)
## group_var effetti casuali
## M_max numero albero, prima ne avevamo 6000, ora per motivi computazionali scendiamo a 300
## inoltre ad ogni step devo stimare un albero con una certa profondità, più depth
## è elevato più ci metterà, quindi metto 2
## lo shrinkage era ottimo a 0.05, ma sappiamo che deve andare assieme al numero di
## alberi e alla profondità. Quindi, dato che fisso 2 come profondità, una scelta per
## bilanciare è di alzare il parametro di shrinkage
#
#save(met_model, file="met_model.Rdata")
#load("met_model.Rdata")
#
#met_model$lmes[[10]]@optinfo$conv$lme4$messages
#
#pdp_land_sf <- metboost_pdp(fit = met_model, data_ref = stima,
#                            var_name = "Size", n_grid = 40)
#plot(pdp_land_sf$x, pdp_land_sf$y, type="l", xlab = "Size",
#     ylab = "y", col = "blue")
#pdp_cond <- metboost_pdp(fit = met_model, data_ref = stima,
#                         var_name = "Condition")
#plot(pdp_cond$x, pdp_cond$y, pch=19)
#
#pred_met <- metboost_predict_manual(met_model, verifica)
#err_met <- mean((verifica$y - pred_met$pred)^2)
#r2_prezzo$metboost <- err_met
#r2_prezzo

cbind(t(r2_prezzo), t(rmse_prezzo), t(mae_prezzo))
```

```
##      [,1]      [,2]      [,3]
## albero 0.08226266 51.76913 37.49173
## RF     0.22756066 47.49455 29.18396
## GBM    0.20150731 48.28888 31.55931
## mert   0.21877669 47.76384 33.24870
## merf   0.29258256 45.45163 28.94816
```